Instituto Tecnológico de Costa Rica

Compiladores e Interpretes

Proyecto #2

Análisis Semántico y Generación de Código Intermedio

Profesor:

Allan Rodríguez Davila

Estudiantes:

Luis Trejos Rivera, 2022437816

Roymar Castillo Carvajal, 2022045111

Semestre II, 2026

2. Manual de usuario

Requisitos

- Java JDK 14 o superior (se necesita el JDK completo, porque App usa javax.tools).
- JFlex 1.9.1, CUP 11b y su runtime. Los .jar vienen en la carpeta tools del proyecto.
- Archivos fuente en UTF-8.

Compilación

Desde la carpeta programa, solo se compilan a mano App y GeneradorAnalizadores. El resto lo genera y compila App automáticamente, solo si faltan archivos o si scanner.flex, parser.cup o algún .java cambió.

```
javac --release 14 -cp "tools\*" -d out src\App.java src\GeneradorAnalizadores.java
```

Ejecución y uso

```
java -cp "out;tools\*" src.App tests\validos\programa_completo.txt
```

También se puede correr desde Visual Studio Code, abriendo App.java y presionando Run (analiza tests/validos/programa_completo.txt). Argumentos opcionales, en este orden: archivo fuente, CSV de tokens (salida/tokens_encontrados.csv), tabla de símbolos (salida/tabla_simbolos.txt) y código intermedio (salida/codigo_intermedio.txt).

- Los errores léxicos, sintácticos y semánticos se muestran en consola con línea y columna.
- Al final se indica si el archivo SI o NO puede ser generado por la gramática.
- El CSV de tokens y la tabla de símbolos se generan siempre. El código intermedio solo si no hubo ningún error.

3. Pruebas de funcionalidad

Archivo	Qué prueba	Resultado
tests/validos/programa_completo.txt	Todos los tipos, arreglos, precedencia, relacionales y lógicas anidadas, if/elif/else, while, for, break, llamadas, read, write, ocultamiento de nombres y funciones declaradas después de principal.	SI. Genera código intermedio.
tests/validos/algoritmos.txt	Recursión, ciclos anidados, mcd, primo, búsqueda y ordenamiento.	SI. Genera código intermedio.
tests/invalidos/errores_semanticos.txt	Un error semántico por línea.	NO. Sin código intermedio.
tests/invalidos/errores_lexicos_sintacticos.txt	Errores léxicos y sintácticos, incluso dentro de funciones y después de principal.	NO. Llega al final del archivo.

3.1 Generación del scanner y del parser

```
PS C:\Users\luist\OneDrive\Escritorio\P2-Compi\programa> java -cp "out;tools\*" src.App tests\validos\programa_completo.txt
Reading "C:\Users\luist\OneDrive\Escritorio\P2-Compi\programa\src\scanner.flex"
Constructing NFA : 283 states in NFA
Converting NFA to DFA :
.....................................................................................................................................
142 states before minimization, 130 states in minimized DFA
Writing code to "generated\Lexer.java"
------- CUP v0.11b 20160615 (GIT 4ac7450) Parser Generation Summary -------
  0 errors and 0 warnings
  53 terminals, 77 non-terminals, and 163 productions declared,
  producing 282 unique parse states.
  0 terminals declared but not used.
  0 non-terminals declared but not used.
  0 productions never reduced.
  0 conflicts detected (0 expected).
  Code written to "Parser.java", and "sym.java".
---------------------------------------------------- (CUP v0.11b 20160615 (GIT 4ac7450))
¡Generación completada con éxito en la carpeta 'generated'!
Note: generated\Parser.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.
--- INICIANDO COMPILADOR ---
Archivo de entrada: tests\validos\programa_completo.txt

Iniciando analisis sintactico...
Reporte de tokens generado en -> salida/tokens_encontrados.csv
Tabla de simbolos generada en -> salida/tabla_simbolos.txt
Codigo intermedio generado en -> salida/codigo_intermedio.txt

RESULTADO: El archivo fuente SI puede ser generado por la gramatica.
PS C:\Users\luist\OneDrive\Escritorio\P2-Compi\programa>
```

Al ejecutar por primera vez el programa, JFlex genera el scanner (Lexer.java) y CUP genera el parser (Parser.java y sym.java). CUP reporta 0 errores, 0 advertencias y 0 conflictos, y el programa compila las clases y ejecuta el análisis.

3.2 Archivo válido: programa_completo.txt

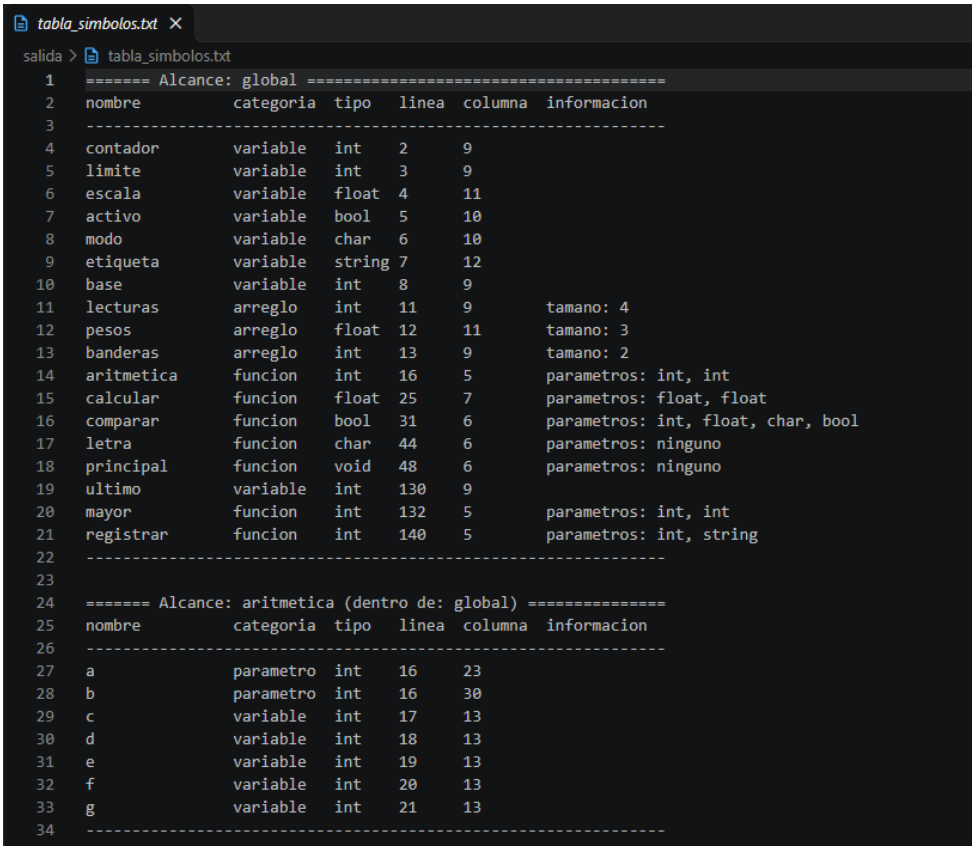
codigo_intermedio.txt

salida > codigo_intermedio.txt

```
1   contador_global = 0
2   i1 = 10
3   limite_global = i1
4   f1 = 2.5
5   escala_global = f1
6   b1 = true
7   activo_global = b1
8   c1 = 'a'
9   modo_global = c1
10  s1 = "sensor"
11  etiqueta_global = s1
12  i2 = 3
13  i3 = 4
14  i4 = i2 + i3
15  base_global = i4
16  i5 = 0
17  i6 = 5
18  lecturas_global[i5] = i6
19  i7 = 1
20  i8 = -2
21  lecturas_global[i7] = i8
22  i9 = 2
23  i10 = 8
24  lecturas_global[i9] = i10
25  i11 = 3
26  i12 = 1
27  lecturas_global[i11] = i12
28  i13 = 0
29  f2 = 0.5
30  pesos_global[i13] = f2
```

Este archivo reúne las funcionalidades del lenguaje: variables globales de todos los tipos, arreglos, funciones, condicionales, ciclos y operaciones de entrada y salida. El programa responde que SI puede ser generado por la gramática y escribe los tres reportes en la carpeta salida.

Código intermedio generado. Las variables globales se escriben primero con el sufijo _global, y cada valor se carga en un temporal (i1, i2, ...) antes de usarse.



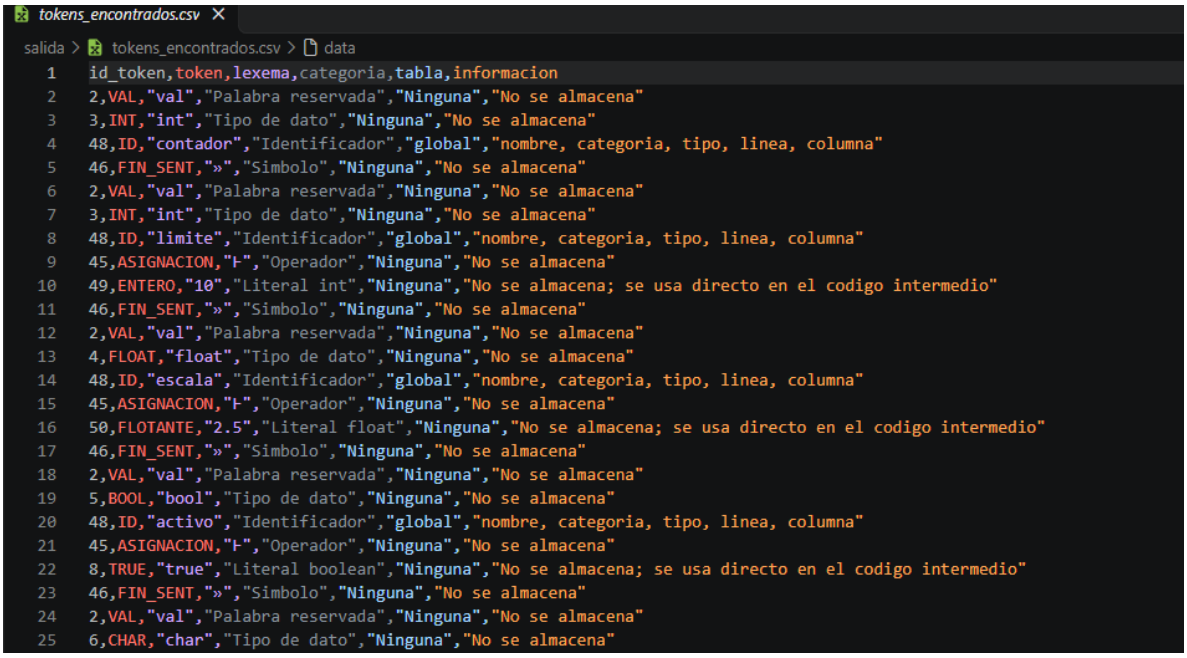
tabla_simbolos.txt

salida > tabla_simbolos.txt

```
======= Alcance: global ====================================
nombre          categoria  tipo   linea  columna  informacion
----------------------------------------------------------------
contador        variable   int    2      9
limite          variable   int    3      9
escala          variable   float  4      11
activo          variable   bool   5      10
modo            variable   char   6      10
etiqueta        variable   string 7      12
base            variable   int    8      9
lecturas        arreglo    int    11     9        tamano: 4
pesos           arreglo    float  12     11       tamano: 3
banderas        arreglo    int    13     9        tamano: 2
aritmetica      funcion    int    16     5        parametros: int, int
calcular        funcion    float  25     7        parametros: float, float
comparar        funcion    bool   31     6        parametros: int, float, char, bool
letra           funcion    char   44     6        parametros: ninguno
principal       funcion    void   48     6        parametros: ninguno
ultimo          variable   int    130    9
mayor           funcion    int    132    5        parametros: int, int
registrar       funcion    int    140    5        parametros: int, string
----------------------------------------------------------------

======= Alcance: aritmetica (dentro de: global) ===============
nombre          categoria  tipo   linea  columna  informacion
----------------------------------------------------------------
a               parametro  int    16     23
b               parametro  int    16     30
c               variable   int    17     13
d               variable   int    18     13
e               variable   int    19     13
f               variable   int    20     13
g               variable   int    21     13
----------------------------------------------------------------
```

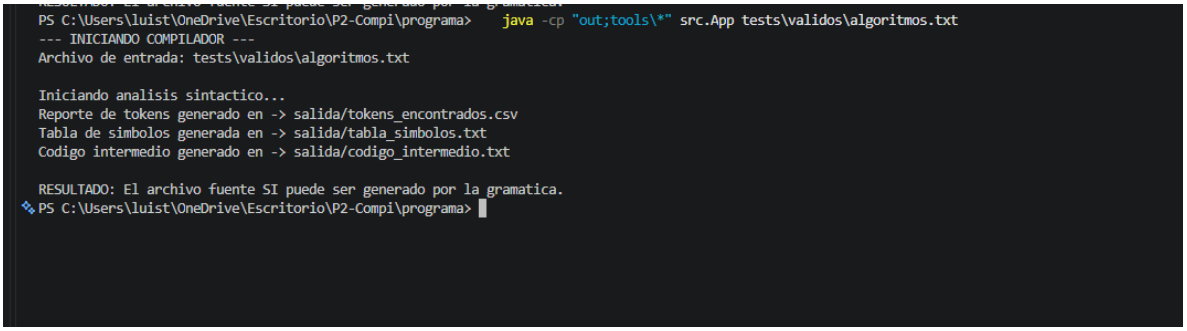
Tabla de símbolos. Está organizada por alcances: el alcance global con sus variables, arreglos y funciones, y un alcance por cada función con sus parámetros y variables locales. Cada símbolo muestra nombre, categoría, tipo, línea y columna.



```
tokens_encontrados.csv
salida > tokens_encontrados.csv > data
id_token,token,lexema,categoria,tabla,informacion
2,VAL,"val","Palabra reservada","Ninguna","No se almacena"
3,INT,"int","Tipo de dato","Ninguna","No se almacena"
48,ID,"contador","Identificador","global","nombre, categoria, tipo, linea, columna"
46,FIN_SENT,"»","Simbolo","Ninguna","No se almacena"
2,VAL,"val","Palabra reservada","Ninguna","No se almacena"
3,INT,"int","Tipo de dato","Ninguna","No se almacena"
48,ID,"limite","Identificador","global","nombre, categoria, tipo, linea, columna"
45,ASIGNACION,"⊢","Operador","Ninguna","No se almacena"
49,ENTERO,"10","Literal int","Ninguna","No se almacena; se usa directo en el codigo intermedio"
46,FIN_SENT,"»","Simbolo","Ninguna","No se almacena"
2,VAL,"val","Palabra reservada","Ninguna","No se almacena"
4,FLOAT,"float","Tipo de dato","Ninguna","No se almacena"
48,ID,"escala","Identificador","global","nombre, categoria, tipo, linea, columna"
45,ASIGNACION,"⊢","Operador","Ninguna","No se almacena"
50,FLOTANTE,"2.5","Literal float","Ninguna","No se almacena; se usa directo en el codigo intermedio"
46,FIN_SENT,"»","Simbolo","Ninguna","No se almacena"
2,VAL,"val","Palabra reservada","Ninguna","No se almacena"
5,BOOL,"bool","Tipo de dato","Ninguna","No se almacena"
48,ID,"activo","Identificador","global","nombre, categoria, tipo, linea, columna"
45,ASIGNACION,"⊢","Operador","Ninguna","No se almacena"
8,TRUE,"true","Literal boolean","Ninguna","No se almacena; se usa directo en el codigo intermedio"
46,FIN_SENT,"»","Simbolo","Ninguna","No se almacena"
2,VAL,"val","Palabra reservada","Ninguna","No se almacena"
6,CHAR,"char","Tipo de dato","Ninguna","No se almacena"
```

Reporte de tokens. El CSV tiene las columnas id_token, token, lexema, categoria, tabla e informacion. Los identificadores indican el alcance donde están guardados, y los literales indican que no se almacenan porque pasan directo al código intermedio.

3.3 Archivo válido: algoritmos.txt



```
PS C:\Users\luist\OneDrive\Escritorio\P2-Compi\programa>    java -cp "out;tools\*" src.App tests\validos\algoritmos.txt
--- INICIANDO COMPILADOR ---
Archivo de entrada: tests\validos\algoritmos.txt

Iniciando analisis sintactico...
Reporte de tokens generado en -> salida/tokens_encontrados.csv
Tabla de simbolos generada en -> salida/tabla_simbolos.txt
Codigo intermedio generado en -> salida/codigo_intermedio.txt

RESULTADO: El archivo fuente SI puede ser generado por la gramatica.
PS C:\Users\luist\OneDrive\Escritorio\P2-Compi\programa>
```

Este archivo prueba programas más completos: recursión, ciclos anidados, mcd, número primo, búsqueda y ordenamiento. El programa lo acepta y genera el código intermedio.

3.4 Errores semánticos: errores_semanticos.txt

Este archivo tiene un error semántico por línea, para probar cada regla del análisis. El programa reporta cada error con su línea y columna, sigue analizando hasta el final del archivo, responde que el archivo NO puede ser generado y no escribe el código intermedio. Entre los errores reportados están:

- Línea 2, columna 9: declaración duplicada ('repetida' ya fue declarado en este alcance).
- Línea 7, columna 27: uso de un identificador antes de declararlo ('tardia').
- Línea 18, columna 5: función sin ningún return ('sin_return').
- Línea 50, columna 11: operación entre tipos incompatibles (el operador '+' no acepta int y float).

- Línea 85, columna 9: llamada con una cantidad incorrecta de argumentos ('parametros' espera 2 y recibe 1).

Se muestra la salida en dos partes; los errores de las líneas 62 a 79 se omiten por espacio. Los errores de código inaccesible aparecen al final de la lista y fuera de orden, porque el programa los detecta cuando cierra el bloque o la función. Las líneas 79 y 80 también se reportan como código inaccesible porque vienen justo después del break y del return de las líneas anteriores, que cortan el flujo de ejecución.

```
PS C:\Users\luist\OneDrive\Escritorio\P2-Compi\programa>    java -cp "out;tools\*" src.App tests\invalidos\errores_semanticos.txt salid
o.txt
--- INICIANDO COMPILADOR ---
Archivo de entrada: tests\invalidos\errores_semanticos.txt

Iniciando analisis sintactico...
Error semantico en la linea 2, columna 9: el identificador 'repetida' ya fue declarado en este alcance
Error semantico en la linea 3, columna 12: el arreglo 'nombres' solo puede ser de tipo int o float
Error semantico en la linea 4, columna 9: el tamano del arreglo 'vacio' debe ser mayor que 0
Error semantico en la linea 5, columna 9: el arreglo 'pocos' tiene tamano 2 pero recibe 3 valores
Error semantico en la linea 6, columna 11: el arreglo 'mezcla' es de tipo float pero sus valores son de tipo int
Error semantico en la linea 7, columna 27: el identificador 'tardia' no ha sido declarado
Error semantico en la linea 10, columna 8: la funcion 'texto' no puede retornar string; solo se permite int, float, char o bool
Error semantico en la linea 14, columna 6: la funcion 'procedimiento' no puede retornar void; solo se permite int, float, char o bool
Error semantico en la linea 18, columna 5: la funcion 'sin_return' no tiene ningun return
Error semantico en la linea 23, columna 5: la funcion 'retorno_incorrecto' debe retornar int, no float
Error semantico en la linea 26, columna 30: el identificador 'a' ya fue declarado en este alcance
Error semantico en la linea 36: codigo inaccesible, la ejecucion nunca llega a esta linea
Error semantico en la linea 47, columna 5: el identificador 'no_existe' no ha sido declarado
Error semantico en la linea 48, columna 16: no se puede asignar un valor de tipo float a 'a1', que es de tipo int
Error semantico en la linea 49, columna 7: no se puede asignar un valor de tipo float a 'i', que es de tipo int
Error semantico en la linea 50, columna 11: el operador '+' no acepta operandos de tipo int y float
Error semantico en la linea 51, columna 11: el operador '/' no acepta operandos de tipo int y int
Error semantico en la linea 52, columna 11: el operador '//' no acepta operandos de tipo float y float
Error semantico en la linea 53, columna 11: el operador 'mod' no acepta operandos de tipo float y float
Error semantico en la linea 54, columna 11: el exponente de 'pot' no puede ser negativo
Error semantico en la linea 55, columna 11: el operador 'pot' no acepta operandos de tipo float y float
Error semantico en la linea 56, columna 11: el operador '+' no acepta operandos de tipo string y string
Error semantico en la linea 57, columna 5: el operador '++' solo se aplica a variables int o float y 'c' es de tipo char
Error semantico en la linea 58, columna 17: el operador '==' no acepta operandos de tipo string y string
Error semantico en la linea 59, columna 17: el operador '<' no acepta operandos de tipo bool y bool
Error semantico en la linea 60, columna 19: el operador '<' no acepta operandos de tipo char y char
Error semantico en la linea 61, columna 17: el operador '==' no acepta operandos de tipo int y float
```

```
Error semantico en la linea 80, columna 14: write solo puede escribir string, int, bool o float y 'c' es de tipo char
Error semantico en la linea 81, columna 13: read solo puede leer variables int, float o char y 'b' es de tipo bool
Error semantico en la linea 82, columna 13: read solo puede leer variables int, float o char y 's' es de tipo string
Error semantico en la linea 84, columna 5: la funcion 'desconocida' no ha sido declarada
Error semantico en la linea 85, columna 9: la funcion 'parametros' espera 2 argumentos y recibe 1
Error semantico en la linea 86, columna 9: el argumento 1 de 'parametros' debe ser de tipo int, no float
Error semantico en la linea 90: codigo inaccesible, la ejecucion nunca llega a esta linea
Error semantico en la linea 80: codigo inaccesible, la ejecucion nunca llega a esta linea
Error semantico en la linea 79: codigo inaccesible, la ejecucion nunca llega a esta linea
Reporte de tokens generado en -> salida\sem_tokens.csv
Tabla de simbolos generada en -> salida\sem_tabla.txt
No se genero el codigo intermedio porque el archivo fuente tiene errores.

RESULTADO: El archivo fuente NO puede ser generado por la gramatica (Errores encontrados).
```

3.5 Errores léxicos y sintácticos: errores_lexicos_sintacticos.txt

Este archivo tiene errores léxicos y sintácticos en líneas separadas, incluso dentro de funciones y después de principal. El programa reporta primero los errores léxicos, luego continúa con el análisis sintáctico y reporta esos también. El análisis llega hasta el final del archivo, responde que el archivo NO puede ser generado y no escribe el código intermedio. Entre los errores reportados están:

- Línea 2, columna 15: carácter no reconocido '@' (léxico).
- Línea 3, columna 9: identificador que comienza con un dígito, '9c' (léxico).

- Línea 41, columna 20: string sin cerrar (léxico).
- Línea 52, columna 1: comentario multilínea sin cerrar (léxico).
- Líneas 33, 36 y 38: falta cerrar la condición con :ɔ (sintáctico).
- Línea 42, columna 22: bloque vacío (sintáctico).

Un error léxico puede causar un error sintáctico justo después, porque el carácter inválido se descarta y el parser recibe una secuencia incompleta. Los mensajes de sintaxis genéricos los genera CUP y están en inglés. Se muestra la salida en dos partes continuas.

```
PS C:\Users\luist\OneDrive\Escritorio\P2-Compi\programa>    java -cp "out;tools\*" src.App tests\invalidos\errores_lexicos_sintacticos.txt salida\l
xt
 --- INICIANDO COMPILADOR ---
 Archivo de entrada: tests\invalidos\errores_lexicos_sintacticos.txt
 Error lexico en linea 2, columna 15: caracter no reconocido '@'
 Error lexico en linea 3, columna 9: identificador no puede comenzar con un digito '9c'
 Error lexico en linea 4, columna 16: caracter no reconocido '.'
 Error lexico en linea 6, columna 15: caracter no reconocido '%'
 Error lexico en linea 7, columna 15: caracter no reconocido '^'
 Error lexico en linea 41, columna 20: string sin cerrar en la misma linea "sin cerrar »                    | string sin cerrar
 Error lexico en linea 52, columna 1: comentario multilinea sin cerrar (falta '!')
 ERROR LEXICO: Se encontraron errores lexicos.

 Iniciando analisis sintactico...
 Error de sintaxis en la linea 1, columna 13 [Lexema problematico: '»'] ==> Syntax error
 instead expected token classes are [TRUE, FALSE, OP_MENOS, OP_NOT, CORCHETE_ABRE, PARENTESIS_ABRE, ID, ENTERO, FLOTANTE, CADENA, CARACTER]
 Error de sintaxis en la linea 2, columna 17 [Lexema problematico: '2'] ==> Syntax error
 instead expected token classes are [OP_SUMA, OP_MENOS, OP_MULT, OP_DIV, OP_DIV_ENT, OP_MOD, OP_POT, FIN_SENT]
 Error de sintaxis en la linea 3, columna 12 [Lexema problematico: 'ⱶ'] ==> Syntax error
 instead expected token classes are [ID]
 Error de sintaxis en la linea 4, columna 17 [Lexema problematico: '0'] ==> Syntax error
 instead expected token classes are [OP_SUMA, OP_MENOS, OP_MULT, OP_DIV, OP_DIV_ENT, OP_MOD, OP_POT, FIN_SENT]
 Error de sintaxis en la linea 5, columna 14 [Lexema problematico: '0'] ==> Syntax error
 instead expected token classes are [OP_SUMA, OP_MENOS, OP_MULT, OP_DIV, OP_DIV_ENT, OP_MOD, OP_POT, FIN_SENT]
 Error de sintaxis en la linea 6, columna 17 [Lexema problematico: '2'] ==> Syntax error
 instead expected token classes are [OP_SUMA, OP_MENOS, OP_MULT, OP_DIV, OP_DIV_ENT, OP_MOD, OP_POT, FIN_SENT]
 Error de sintaxis en la linea 7, columna 17 [Lexema problematico: '2'] ==> Syntax error
 instead expected token classes are [OP_SUMA, OP_MENOS, OP_MULT, OP_DIV, OP_DIV_ENT, OP_MOD, OP_POT, FIN_SENT]
 Error de sintaxis en la linea 8, columna 13 [Lexema problematico: 'n'] ==> Syntax error
 instead expected token classes are [ENTERO]
 Error de sintaxis en la linea 9, columna 24 [Lexema problematico: '+'] ==> Syntax error
 instead expected token classes are [LLAVE_CIERRA, COMA]
 Error de sintaxis en la linea 9, columna 34 [Lexema problematico: '»'] ==> Syntax error
 instead expected token classes are [error, VAL, INT, FLOAT, BOOL, CHAR, STR, VOID]
 Error de sintaxis en la linea 11, columna 23 [Lexema problematico: ':ɔ'] ==> Syntax error
 instead expected token classes are [ID]
 Error de sintaxis en la linea 16, columna 17 [Lexema problematico: '»'] ==> Syntax error
 instead expected token classes are [TRUE, FALSE, OP_MENOS, OP_NOT, CORCHETE_ABRE, PARENTESIS_ABRE, ID, ENTERO, FLOTANTE, CADENA, CARACTER]
 Error de sintaxis en la linea 22, columna 18 [Lexema problematico: 'x'] ==> Syntax error
 instead expected token classes are [ENTERO, FLOTANTE]
 Error de sintaxis en la linea 23, columna 10 [Lexema problematico: '++'] ==> Syntax error
 instead expected token classes are [OP_SUMA, OP_MENOS, OP_MULT, OP_DIV, OP_DIV_ENT, OP_MOD, OP_POT, FIN_SENT]
```

```
 Error de sintaxis en la linea 24, columna 5 [Lexema problematico: '++'] ==> Syntax error
 instead expected token classes are [error, VAL, IF, WHILE, FOR, RETURN, BREAK, WRITE, READ, LLAVE_ABRE, LLAVE_CIERRA, ID]
 Error de sintaxis en la linea 25, columna 7 [Lexema problematico: '+'] ==> Syntax error
 instead expected token classes are [OP_INC, OP_DEC, CORCHETE_ABRE, PARENTESIS_ABRE, ASIGNACION]
 Error de sintaxis en la linea 26, columna 21 [Lexema problematico: '<'] ==> Syntax error
 instead expected token classes are [OP_SUMA, OP_MENOS, OP_MULT, OP_DIV, OP_DIV_ENT, OP_MOD, OP_POT, FIN_SENT]
 Error de sintaxis en la linea 27, columna 37 [Lexema problematico: '<'] ==> Syntax error
 instead expected token classes are [FIN_SENT]
 Error de sintaxis en la linea 28, columna 24 [Lexema problematico: 'λ'] ==> Syntax error
 instead expected token classes are [OP_SUMA, OP_MENOS, OP_MULT, OP_DIV, OP_DIV_ENT, OP_MOD, OP_POT, FIN_SENT]
 Error de sintaxis en la linea 29, columna 16 [Lexema problematico: '+'] ==> Syntax error
 instead expected token classes are [PARENTESIS_CIERRA]
 Error de sintaxis en la linea 30, columna 15 [Lexema problematico: ','] ==> Syntax error
 instead expected token classes are [PARENTESIS_CIERRA]
 Error de sintaxis en la linea 31, columna 14 [Lexema problematico: 'a'] ==> Syntax error
 instead expected token classes are [TRUE, FALSE, OP_MENOS, ID, ENTERO, FLOTANTE, CADENA]
 Error de sintaxis en la linea 32, columna 15 [Lexema problematico: '+'] ==> Syntax error
 instead expected token classes are [PARENTESIS_CIERRA]
 Error de sintaxis en la linea 33, columna 11 ==> falta cerrar la condicion con :ɔ
 Error de sintaxis en la linea 36, columna 8 ==> falta cerrar la condicion con :ɔ
 Error de sintaxis en la linea 38, columna 13 ==> falta cerrar la condicion con :ɔ
 Error de sintaxis en la linea 42, columna 5 [Lexema problematico: 'if'] ==> Syntax error
 instead expected token classes are [TRUE, FALSE, OP_MENOS, OP_NOT, CORCHETE_ABRE, PARENTESIS_ABRE, ID, ENTERO, FLOTANTE, CADENA, CARACTER]
 Error de sintaxis en la linea 42, columna 22 [Lexema problematico: ':?'] ==> Syntax error
 instead expected token classes are [error, VAL, IF, WHILE, FOR, RETURN, BREAK, WRITE, READ, LLAVE_ABRE, ID]
 Error de sintaxis en la linea 46, columna 19 [Lexema problematico: '»'] ==> Syntax error
 instead expected token classes are [TRUE, FALSE, OP_MENOS, OP_NOT, CORCHETE_ABRE, PARENTESIS_ABRE, ID, ENTERO, FLOTANTE, CADENA, CARACTER]
 Reporte de tokens generado en -> salida\lex_tokens.csv
 Tabla de simbolos generada en -> salida\lex_tabla.txt
 No se genero el codigo intermedio porque el archivo fuente tiene errores.

 RESULTADO: El archivo fuente NO puede ser generado por la gramatica (Errores encontrados).
```

4. Descripción del problema

El proyecto consiste en desarrollar las fases de análisis semántico y de generación de código intermedio para el lenguaje imperativo ligero, orientado a la configuración de chips en sistemas empotrados, que el equipo definió en la Tarea #1 y para el cual se implementó el análisis léxico y sintáctico en el Proyecto #1.

A partir del scanner (JFlex) y del parser (CUP) del Proyecto #1, el programa recibe un archivo fuente y realiza lo siguiente:

- Preserva y corrige los alcances del Proyecto #1.
- Indica si el archivo fuente puede o no ser generado por la gramática, tomando en cuenta la gramática, la sintaxis y la semántica (tipado explícito y fuerte).
- Maneja los errores léxicos, sintácticos y semánticos con la técnica de recuperación en modo pánico: cada error se reporta con su línea y columna y el análisis continúa.
- Escribe en un archivo el código intermedio de tres direcciones, con el mismo sentido semántico que el archivo fuente.

Todo programa debe tener exactamente una función principal. Además del código intermedio, el programa genera un reporte CSV con los tokens encontrados y un reporte de la tabla de símbolos organizado por alcances.

El análisis semántico se implementó con acciones semánticas de CUP que llaman a clases Java propias (tabla de símbolos, reglas de tipos y generador de código). La validación semántica y la generación del código intermedio se realizan en una sola pasada del parser.

5. Diseño del programa

Arquitectura general

Componente	**Descripción**
scanner.flex y Lexer.java	Definen y generan el scanner con JFlex. Reconocen los tokens, ignoran comentarios y reportan errores léxicos con línea y columna.
parser.cup, Parser.java y sym.java	Definen y generan el parser LALR(1) con CUP. La gramática lleva las acciones que llaman al análisis semántico y al generador de código.
Analizador	Coordina las fases: lee los tokens, ejecuta el parser, escribe los reportes y decide si se escribe el código intermedio.
App y GeneradorAnalizadores	Generan los analizadores con JFlex y CUP solo cuando hay cambios, compilan el proyecto y ejecutan Analizador.
tabla.Simbolo, SimboloArreglo, SimboloFuncion	Representan una variable o parámetro, un arreglo (con tamaño) y una función (con los tipos de sus parámetros).
tabla.Alcance y TablaSimbolos	Agrupan los símbolos por alcance y manejan la pila de alcances abiertos. Escriben el reporte de la tabla de símbolos.
tabla.ReporteTokens	Escribe el CSV de tokens e indica en qué tabla (alcance) está guardado cada identificador.
semantico.AnalizadorSemantico	Contiene todas las comprobaciones semánticas y reporta los errores con línea y columna.
semantico.RecolectorFunciones	Recorre los tokens antes del parser para registrar la firma de cada función.
semantico.ResultadoExpresion y ResultadoSentencia	Atributos que devuelven las reglas del parser: tipo, dirección y código de una expresión; código de una línea y si corta el flujo.
codigo.GeneradorCodigo y EncabezadoFor	Generan el código de tres direcciones y llevan los contadores de temporales y de etiquetas.

Resumen del lenguaje

Elemento	Forma
Bloque	¿: ... :?
Paréntesis	є: ... :ɔ
Índice de arreglo	∫: ... :ɳ
Asignación y fin de sentencia	⊢ y »
Comentarios	De línea: | texto. Multilínea: ¡ texto !
Tipos	int, float, bool, char, string
Declaración	val int x ⊢ 5 » o val int x »
Arreglo	val int a ∫:3:ɳ ⊢ ¿: 1, 2, 3 :? » o val int a ∫:3:ɳ »
Función	int suma є: int a, int b :ɔ ¿: return a + b » :?
Función principal	void principal є: :ɔ ¿: ... :?
Operadores aritméticos	+, -, *, /, //, mod, pot, ++, --
Operadores relacionales	∫: a :ɳ < ∫: b :ɳ con <, >, <=, >=, ==, !=
Operadores lógicos	є: a :ɔ λ є: b :ɔ, є: a :ɔ θ є: b :ɔ, Σ є: a :ɔ
Condicionales	if є: cond :ɔ ¿: ... :? elif ... else ...
Ciclos	while є: cond :ɔ ¿: ... :? y for є: val int i ⊢ 0 » ∫: i :ɳ < ∫: 5 :ɳ » i++ :ɔ ¿: ... :?
Entrada y salida	read є: x :ɔ » y write є: x :ɔ »

Algoritmos usados

- Traducción dirigida por la sintaxis: cada expresión devuelve un ResultadoExpresion con tipo, dirección y código; cada línea un ResultadoSentencia con su código y si corta el flujo. Un error semántico da el tipo error, que evita errores en cascada.
- Tabla de símbolos: pila de alcances abiertos (global, función, bloque y for) más una lista de todos para el reporte. Buscar un nombre sube desde el alcance actual hasta el global; los duplicados se buscan solo en el alcance actual.
- Precedencia: jerarquía de reglas en CUP. De mayor a menor: pot (asociativo a la derecha), + -, * / //, mod. Los demás son asociativos a la izquierda.
- Modo pánico: el scanner descarta el carácter o lexema inválido y sigue. El parser usa producciones con error que sincronizan en », en :? y en :ɔ. Se agregó una producción de error específica para una condición sin :ɔ (producción de errores).

Reglas semánticas principales

Aspecto	**Regla**
Declaraciones	No se repite un nombre en el mismo alcance (un local puede ocultar a uno exterior). Todo identificador se declara antes de usarse.
Operadores	+ - *: dos int o dos float. /: float. // mod pot: int (exponente de pot no negativo). Relacionales dan bool; == != aceptan también bool y char. λ θ Σ solo bool. No hay conversión entre int y float.
Condiciones y arreglos	Las condiciones son bool. Los arreglos son int o float, con tamaño mayor que 0, índice int y no más valores que el tamaño.

Funciones	Retornan int, float, char o bool, con al menos un return del tipo correcto. principal no retorna. Una llamada valida existencia, cantidad y tipos de argumentos.
Otros	break solo dentro de while o for. read acepta int, float y char. write acepta string, int, bool y float. Una línea después de return o break es código inaccesible (error).

Código intermedio

Código de tres direcciones. Los temporales llevan una letra por tipo (i, f, b, c, s), las etiquetas un número único (if_true_1, while_4, for_6) y las funciones se encierran entre tipo_nombre: y tipo_nombre_end. Las globales llevan el sufijo _global. Ejemplo, la función mayor:

```
1. int_mayor:
2. param_int a
3. param_int b
4. i145 = a
5. i146 = b
6. b41 = i145 > i146
7. if b41 goto if_true_9
8. goto if_false_9
9. if_true_9:
10. i147 = a
11. return i147
12. goto if_end_9
13. if_false_9:
14. i148 = b
15. return i148
16. if_end_9:
17. int_mayor_end
```

6. Librerías usadas

- JFlex 1.9.1: genera Lexer.java a partir de scanner.flex.
- CUP 11b y su runtime: generan Parser.java y sym.java a partir de parser.cup, y el runtime es necesario para ejecutar el parser.
- Java SE 14 o superior: java.io y java.nio.file para leer el fuente y crear los reportes en UTF-8, java.util para las estructuras, java.math.BigDecimal para los literales flotantes y javax.tools con ProcessBuilder para compilar y ejecutar desde App.

No se usaron librerías adicionales.

7. Análisis de resultados

Objetivos alcanzados

- Analizador semántico con tipado explícito y fuerte y tabla de símbolos por alcances.
- Errores léxicos, sintácticos y semánticos con línea y columna, con recuperación en modo pánico (el código inaccesible indica solo la línea).
- Indicación de si el archivo puede o no ser generado por la gramática.

- Código intermedio de tres direcciones para expresiones, arreglos, if/elif/else, while, for, break, funciones, read y write.
- Correcciones de la revisión del Proyecto #1 y una producción de error específica (punto extra de recuperación).

Objetivos no alcanzados

- Todos los objetivos planteados fueron alcanzados satisfactoriamente.

Limitaciones conocidas

- Un literal flotante no puede terminar en cero, salvo 0.0 (regla del scanner del Proyecto #1). Por ejemplo 2.0 no es válido.
- Se comprueba que una función tenga return, pero no que todos los caminos terminen en uno.
- Los mensajes de sintaxis de CUP están en inglés, y un error léxico puede causar un error sintáctico justo después.

8. Justificación de toma de decisiones

Cómo se realizó la validación semántica

Se usó traducción dirigida por la sintaxis con atributos sintetizados, como en el material del curso, para validar y generar código en una sola pasada. Cada regla de CUP calcula su atributo a partir de sus partes y llama a AnalizadorSemantico (valida) y a GeneradorCodigo (genera). La lógica está en clases Java separadas para que el .cup quede centrado en la gramática y para que dos personas trabajen sin conflictos en git. Un error se reporta al detectarse y la expresión toma el tipo error. Un símbolo con error se agrega igual a la tabla para no causar errores de "no declarado" después.

Estructuras de datos

Estructura	**Por qué**
Stack<Alcance> y lista de alcances	Los alcances se abren y cierran en orden de anidamiento. La lista conserva los cerrados para el reporte.
Alcance con referencia al padre	Subir por los padres encuentra siempre la declaración más cercana.
Subclases de Simbolo	Un arreglo guarda su tamaño y una función los tipos de sus parámetros.
HashMap de funciones	Búsqueda directa por nombre al validar una llamada, aunque la función esté más abajo.
ResultadoExpresion y ResultadoSentencia	Llevan tipo, dirección y código, y si una línea corta el flujo (código inaccesible).
Pilas de números de etiqueta	Las estructuras anidadas no mezclan sus etiquetas y break sabe a qué ciclo salir.

Decisiones de diseño

Decisión	**Justificación**
No se mezclan int y float; Solo float y // solo int	Tipado explícito y fuerte: ninguna conversión automática.
Un nombre local puede ocultar a uno exterior	Como en C. Solo se prohíbe repetir en el mismo alcance.

Funciones en cualquier orden, una sola principal	Declarar una función no ejecuta nada, así que se puede llamar antes. Una global debe declararse antes de usarse.
Una función debe tener al menos un return	Revisar todos los caminos requiere análisis de flujo fuera del alcance del proyecto.
Arreglos con menos valores que su tamaño se rellenan	El profesor permitió rellenar o dar error; se rellena con el valor por defecto. Más valores que el tamaño sí es error.
El código inaccesible es un error	Sugerencia del profesor, igual que en Java.
Sufijos _global y _local	Evitan que dos variables con el mismo nombre se confundan en el código intermedio.

Correcciones al Proyecto #1

Según la revisión del Proyecto #1: las operaciones lógicas y ==/!= aceptan paréntesis y anidamiento, la primera sección del for acepta una declaración o una asignación, se permiten arreglos sin asignación y bloques anidados, se implementó la tabla de símbolos y App solo genera y compila cuando hay cambios. Además, str, % y ^ pasaron a string, mod y pot.

9. Bitácora

Commits en el repositorio de GitHub: https://github.com/LuisK19/P2-Compi